

Crudo

Crudo lets you take slices from raw blocks and more: slice, filter, transform and join the lines of raw blocks. *Crudo* can also extract named regions out of source files, to display them without having to hardcode linenumbers.

v0.1.1 September 28, 2024

<https://github.com/SillyFreak/typst-crudo>

Clemens Koza

CONTENTS

I	Introduction	2
I.a	raw elements as lists of lines	2
I.b	Handling regions in code files	2
I.b.a	Highlighting regions	4
I.c	Documenting evolving code snippets . .	5
I.c.a	Highlighting regions inside evolving code snippets	7
II	Module reference	8
II.a	regions	13
II.b	history	15

I INTRODUCTION

raw elements display source code line by line, but Typst doesn't provide a lot of convenience for handling them in this way: you need to extract the `raw.text` to do anything with it, and then transform it back. But also if you read in a source file for displaying code, manipulating the resulting string as multiple lines is not too convenient. *Crudo* gives you a few extra tools to make this easier.

I.a raw elements as lists of lines

raw elements feel like lists of lines; it's common to want to extract specific lines, join multiple ones together, etc. As values, though, raw elements don't behave this way.

While a package can't add methods such as `raw.slice()` to an element, we can at least provide functions to help with common tasks. The module reference describes these utility functions:

- `r2l()` and `l2r()` are the building blocks the others build on: *raw-to-lines* and *lines-to-raw* conversions.
- `transform-text()` and `transform()` are one layer above and allow arbitrarily transforming the text content, viewed as a single string or an array of strings, respectively.
- `read()` reads a text file and puts it in a raw element with the provided raw properties.
- `map()`, `filter()` and `slice()` are analogous to their array counterparts.
- `lines()` is similar to `slice()` but allows more advanced line selections in a single step.
- `join()` combines multiple raw elements and is convenient e.g. to add preambles to code snippets.

All functions that accept raw elements as parameters alternatively accept simple strings. In these cases, a string code behaves like `raw(code)`, i.e. it's not a block element and has no `lang` set on it. This is mostly useful with `join()`, which takes multiple raw elements, but the other functions don't disallow this usage.

I.b Handling regions in code files

Crudo also contains utilities for identifying and extracting ranges of code from source files. Let's assume you have the following source file (any language will work; we're not using Typst for the example to avoid making things too meta):

```
1 // @region start:basics java
2 public class Account {
3     private double balance;
4
5     public Konto(double balance) {
6         this.balance = balance;
7     }
8
9     // @region end:basics start:mutators
10    public void deposit(double amount) {
11        this.balance += amount;
12    }
13
14    public boolean withdraw(double amount) {
15        // @region start:withdraw-body
16        if(this.balance - amount >= 0) {
17            this.balance -= amount;
18            return true;
```

```

19   }
20   return false;
21 // @region end:withdraw-body
22   }
23
24 // @region end:mutators start:basics
25   public double getBalance() {
26       return this.balance;
27   }
28 }
29 // @region end:basics

```

You can already see that this file contains a few special comments of the form `// @region ...`; these can be processed by *Crudo* to identify and extract ranges; for example, using the `regions.ranges()` function:

<pre> 1 #let code = crudo.read("Account.java", typ 2 properties: (block: true, lang: "java")) 3 #crudo.regions.ranges(code, "basics") \ 4 #crudo.regions.ranges(code, "mutators") \ 5 #crudo.regions.ranges(code, 6 ("mutators", "!withdraw-body")) </pre>	<pre> ((2, 9), (25, 29)) ((10, 15), (16, 21), (22, 24)) ((10, 15), (22, 24)) </pre>
--	---

The result of `regions.ranges()` is an array of region bounds; for example, the `(2, 9)` means that lines 2 (inclusive) to 9 (exclusive) are part of the basics region. A leading `!` can be used to exclude a region. Region indicators that may be embedded in a region are of course skipped, e.g. lines 15 and 21 for `withdraw-body` inside the `mutators` region.

These ranges can be further processed for use with `lines()` or packages such as `codly` or `zebraw`:

```

1 #import "@preview/codly:1.3.0"
2 #import "@preview/zebraw:0.6.1"
3 #let ranges = crudo.regions.ranges(code, "basics")
4 #grid(
5   columns: (lfr, lfr, lfr),
6   crudo.lines(code, ..ranges.map((a, b) => range(a, b))),
7   codly.codly-init[
8     #codly.codly(ranges: ranges.map((a, b) => (a, b - 1)))
9     #code
10  ],
11   zebraw.zebraw(line-range: ranges, code),
12 )

```

```

public class Account {
  private double balance;

  public Konto(double balance) {
    this.balance = balance;
  }

  public double getBalance() {
    return this.balance;
  }
}

```

```

2 public class Account {
3   private double balance;
4
5   public Konto(double balance) {
6     this.balance = balance;
7   }
8
9   public double getBalance() {
10    return this.balance;
11  }
12 }

```

```

2 public class Account {
3   private double balance;
4
5   public Konto(double balance) {
6     this.balance = balance;
7   }
8
9   : 16 lines skipped
10  public double getBalance() {
11    return this.balance;
12  }
13 }

```

There are some differences in the exact parameters:

- `lines()` expects arrays of individual lines (which we can construct using the `range()` function). This exact usage can be simplified by using the `regions.extract()` shorthand;
- `codly` uses an inclusive upper bound for ranges;
- `zebraw` uses exclusive upper bounds, so we don't need to change the ranges at all.

An important difference between the first and the latter two examples is line numbering: if we used one of the two code block libraries to format the first code snippet, we'd see line numbers from 1 to 11. This is because we are producing a new raw element with just a subset of the lines, and any subsequent formatting is not aware of the missing lines. In contrast, the latter two examples keep the original raw element, and instruct the library to only render some of those. The libraries are aware of the actual line numbers and will preserve them.

I.b.a Highlighting regions

Identifying region ranges can of course not only be used to filter lines, but also for highlighting certain ranges. Here is again an example using both `codly` and `zebraw`:

```
1 #let ranges = crudo.regions.ranges(code, "mutators")
2 #let highlights = crudo.regions.ranges(code, "withdraw-body")
3 #let highlight-lines = highlights.map((a, b) => range(a, b)).join()
4 #grid(
5   columns: (lfr, lfr),
6   codly.codly-init[
7     #codly.codly(ranges: ranges.map((a, b) => (a, b - 1)),
8       highlighted-lines: highlight-lines)
9     #code
10  ],
11  zebraw.zebraw(line-range: ranges, highlight-lines: highlight-lines, code),
12 )
```

typ

```
10 public void deposit(double amount) {
11     this.balance += amount;
12 }
13
14 public boolean withdraw(double amount) {
16     if(this.balance - amount >= 0) {
17         this.balance -= amount;
18         return true;
19     }
20     return false;
22 }
23
```

java

```
10 public void deposit(double amount) {
11     this.balance += amount;
12 }
13
14 public boolean withdraw(double amount) {
16     if(this.balance - amount >= 0) {
17         this.balance -= amount;
18         return true;
19     }
20     return false;
22 }
23
```

java

As before, the displayed lines are limited to a specific region; a subregion is additionally highlighted. Both libraries take an array of line numbers for highlights. We create that by applying `range()` to each highlighted range, and joining all such ranges into a single array.

If you instead want to use the `lines()` approach (to not have gaps in the line numbers), there's a little more work to do: the highlighted range is of the numbers (16, 21) but the raw block returned by `lines()` will not contain these lines (or have different code in these lines). The `regions.ranges-within()` function can handle this situation for you: it transforms a list of line number ranges into what they should be inside a snippet that has some lines skipped. Below is an example:

```

1 #let ranges = crudo.regions.ranges(code, "mutators")
2 #let highlights = crudo.regions.ranges-within(ranges,
3   crudo.regions.ranges(code, "withdraw-body"))
4 #let highlight-lines = highlights.map((a, b) => range(a, b)).join()
5 #codly.codly-init[
6   #codly.codly(highlighted-lines: highlight-lines)
7   #crudo.lines(code, ..ranges.map((a, b) => range(a, b)))
8 ]

```

```

1 public void deposit(double amount) {
2     this.balance += amount;
3 }
4
5 public boolean withdraw(double amount) {
6     if(this.balance - amount >= 0) {
7         this.balance -= amount;
8         return true;
9     }
10    return false;
11 }

```

I.c Documenting evolving code snippets

For explanatory texts it is often useful to develop a code snippet in multiple steps, where early code is later replaced by a more capable version.¹ Using the region feature shown above for that can quickly get out of hand. Consider this code as an example:

```

1 pub fn main() {
2     // @region start:simple
3     println!("Hello World");
4     // @region end:simple start:parametric
5     let name = todo!();
6     println!("Hello {name}");
7     // @region end:parametric start:localized
8     let local_greeting = todo!();
9     let name = todo!();
10    println!("{local_greeting} {name}");
11    // @region end:localized
12 }

```

This code file doesn't really consist of multiple parts that are explained separately, but different stages of development. The `main()` function declaration should always be presented, and individual parts of the implementation should be shown. To do so, the example on the next page always skips *all but one* of the implementation snippets. It gets worse when there are multiple parts of the code that evolve: the set of regions will grow, and which regions belong together becomes increasingly complex. Through this complexity, the advantage of not having to hardcode line numbers vanishes.

An additional downside is this: when executed, it would run *all parts of the logic* (the `todo!()`s notwithstanding), not just the latest one. Not even the final form of the code can be tested!

¹In particular, this feature is inspired by *Crafting Interpreters* by Robert Nystrom. For example, in "Executing statements" right at the end, the `interpreter.interpret(statements);` line replaces a previous one. In the accompanying source code on Github, you can see how both the old and new source code is modelled using history-aware regions.

```

1 #let code = crudo.read("greet.rs", properties: (block: true, lang: "rust")) typ
2 #grid(
3   columns: (1fr, 1.2fr),
4   crudo.regions.extract(code, ("!parametric", "!localized")) +
5   crudo.regions.extract(code, ("!simple", "!localized")),
6   crudo.regions.extract(code, ("!simple", "!parametric")),
7 )

```

```

1 pub fn main() { rust
2   println!("Hello World");
3 }

```

```

1 pub fn main() { rust
2   let name = todo!();
3   println!("Hello {name}");
4 }

```

```

1 pub fn main() { rust
2   let local_greeting = todo!();
3   let name = todo!();
4   println!("{local_greeting} {name}");
5 }

```

To handle this kind of use case, *Crudo* provides the history module. The `history.ranges()` and `history.extract()` functions work similar to their region counterparts, but instead of taking any number of regions, they take the single current step and a complete list of the steps in the file's history as parameters. The `history.ranges-within` function is a direct alias to `regions.ranges-within()`, as it's directly applicable to the history module as well.

Here's how to use the module for a direct comparison:

```

1 #let code = crudo.read("greet.rs", properties: (block: true, lang: "rust")) typ
2 #let history = ("simple", "parametric", "localized")
3 #grid(
4   columns: (1fr, 1.2fr),
5   crudo.history.extract(code, "simple", history) +
6   crudo.history.extract(code, "parametric", history),
7   crudo.history.extract(code, "localized", history),
8 )

```

```

1 pub fn main() { rust
2   println!("Hello World");
3 }

```

```

1 pub fn main() { rust
2   let name = todo!();
3   println!("Hello {name}");
4 }

```

```

1 pub fn main() { rust
2   let local_greeting = todo!();
3   let name = todo!();
4   println!("{local_greeting} {name}");
5 }

```

And here is how the code is prepared to be used with the history module:

```

1 // @start:simple rust
2 pub fn main() {
3   /* @before:parametric
4   println!("Hello World");
5   */
6   // @start:localized
7   // @region start:TODOs

```

```

8     let local_greeting = todo!();
9     // @end:localized
10    // @start:parametric
11    let name = todo!();
12    // @region end:TODOs
13    /* @before:localized
14    println!("Hello {name}");
15    */
16    // @end:parametric
17    // @start:localized
18    println!("{local_greeting} {name}");
19    // @end:localized
20 }
21 // @end:simple

```

There are some differences here:

- We are not declaring regions but steps; instead of writing `@region start:` we're just writing `@start:.` Each tag can also only start or end a single step, not multiple or both, and steps must be properly nested to reflect them happening sequentially.
 - There is also a region called `TODOs`; you can learn about that in Section I.c.a.
- The code as a whole is enclosed in a step. This is necessary: code outside a step would not be considered part of the history and wouldn't be picked up.
- There are also `@before:` tags, and these are wrapped in block comments. That means when running the annotated code, only the final version of the code is actually used. A `@before` tag's content will be removed from the enclosing step when the declared step is reached. For example, line 12 in the `@before:localized` tag will be displayed in the `parametric` step, but not later.

You may have also noticed that the `let name = todo!();` line is no longer duplicated; achieving that with regions would have been way harder.

That `@before` tags are block comments also has its downsides, unfortunately: when not using `extract` and instead using a code block library to limit the lines, the syntax highlighting will pick up on the code actually being a comment:

```

1 #let ranges = crudo.history.ranges( typ
2     code, "simple", history)
3 #codly.codly(ranges: ranges.map(
4     ((a, b)) => (a, b - 1)))
5 #code

```

```

2 pub fn main() { rust
4     println!("Hello World");
20 }

```

The history module is therefore less useful for this kind of usage.

I.c.a Highlighting regions inside evolving code snippets

While using histories is great, they don't readily support selecting regions for highlighting: you can't easily select *only* the current step, or a different subset of lines. The `history.ranges()` function therefore slightly integrates with `regions.ranges()`, in that it *also* ignores region indicators. That lets you insert region indicators into your code files which won't be shown in code snippets, which you can then use for highlighting:

```

1 #let history = ("simple", "parametric", "localized")
2 #let ranges = crudo.history.ranges(code, "parametric", history)
3 #let highlights = crudo.regions.ranges(code, "TODOs")
4 #let highlight-lines = highlights.map((a, b) => range(a, b)).join()
5 #grid(
6   columns: (1fr, 1fr),
7   codly.codly-init[
8     #codly.codly(ranges: ranges.map((a, b) => (a, b - 1)),
9       highlighted-lines: highlight-lines)
10    #code
11  ],
12  zebraw.zebraw(line-range: ranges, highlight-lines: highlight-lines, code),
13 )

```

```

2 pub fn main() {
11   let name = todo!();
14   println!("Hello {name}");
20 }

```

```

2 pub fn main() {
: 8 lines skipped
11   let name = todo!();
: 2 lines skipped
14   println!("Hello {name}");
: 5 lines skipped
20 }

```

The TODOs range contained more than just this one line (another one comes from the localized step), but the non-shown lines don't hurt during highlighting. Likewise, `history.ranges-within` will ignore them as well.

II MODULE REFERENCE

- `r2l()`
- `l2r()`
- `transform-text()`
- `transform()`

- `read()`
- `map()`
- `filter()`
- `slice()`

- `lines()`
- `join()`

`r2l(raw-block: content str) -> array`

raw-to-lines: extract lines and properties from a raw element.

```

1 crudo.r2l(``txt
2 first line
3 second line
4 ``)

```

```

(
  ("first line", "second line"),
  (block: true, lang: "txt"),
)

```

Note that even though you will usually want to use this on raw *blocks*, this is not a necessity:

```

1 crudo.r2l(
2   raw("first line\nsecond line")
3 )

```

```
((("first line", "second line"), (:))
```

For flexibility, regular strings are also supported. Strings don't have a language and aren't blocks:

```
1 crudo.r2l("first line\nsecond line")
```

```
((("first line", "second line"), (:))
```


Parameters:

`raw-block (content or str)` – a single raw element or (multi line) string

```
l2r(lines: array, ..properties: arguments) -> content
```

lines-to-raw: convert lines into a raw element. Properties for the created element can be passed as parameters.

<pre>1 crudo.l2r(2 ("first line", "second line") 3)</pre>	typc <pre>first line second line</pre>
---	---

Note that even though you will usually want to construct raw *blocks*, this is not assumed. To create blocks, pass the appropriate parameter:

<pre>1 crudo.l2r(2 ("first line", "second line"), 3 block: true, 4)</pre>	typc <pre>first line second line</pre>
---	---

Parameters:

`lines (array)` – an array of strings

`..properties (arguments)` – properties for constructing the new raw element

```
transform-text(raw-block: content str, mapper: function) -> content
```

Transforms the text of a raw element and creates a new one with the new text. All properties of the element (e.g. block and lang) are preserved.

<pre>1 crudo.transform-text(2 ```typc 3 4 let foo() = { 5 // some comment 6 ... do something ... 7 } 8 ```, 9 str.trim 10)</pre>	typc <pre>let foo() = { // some comment ... do something ... }</pre>
--	---

Parameters:

`raw-block (content or str)` – a single raw element or (multi line) string

`mapper (function)` – a function that takes a single string and returns a new one

```
transform(raw-block: content str, mapper: function) -> content
```

Transforms all lines of a raw element and creates a new one with the lines. All properties of the element (e.g. block and lang) are preserved.

```
1  crudo.transform(typc
2    ```typc
3    let foo() = {
4      // some comment
5      ... do something ...
6    }
7    ``` ,
8    lines => lines.filter(l => {
9      // only preserve non-comment lines
10     not l.starts-with(regex("\s*//"))
11   })
12 )
```

```
let foo() = {
  ... do something ...
}
```

Parameters:

raw-block (content or str) – a single raw element or (multi line) string

mapper (function) – a function that takes an array of strings and returns a new one

```
read(properties: dictionary, trim: boolean alignment, ..args: arguments) -> content
```

A wrapper around the built-in read() function that returns the file contents as a raw element. Since code files often have a trailing newline by convention, this function can optionally trim the file contents (and trims the end by default).

Parameters:

properties (dictionary = (:)) – properties for constructing the new raw element, given as a dictionary instead of direct arguments since the latter is used for the read() parameters

trim (boolean or alignment = end) – one of true, false, start, end to determine whether and what to trim() from the read file

..args (arguments) – the parameters to read(), i.e. file name and encoding

```
map(raw-block: content str, mapper: function) -> content
```

Maps individual lines of a raw element and creates a new one with the lines. All properties of the element (e.g. block and lang) are preserved.

```

1 crudo.map(
2   ```typc
3   let foo() = {
4     // some comment
5     ... do something ...
6   }
7   ```,
8   l => l.trim()
9 )

```

```

let foo() = {
  // some comment
  ... do something ...
}

```

Parameters:

raw-block (content or str) – a single raw element or (multi line) string

mapper (function) – a function that takes a string and returns a new one

```
filter(raw-block: content str, test: function) -> content
```

Filters lines of a raw element and creates a new one with the lines. All properties of the element (e.g. block and lang) are preserved.

```

1 crudo.filter(
2   ```typc
3   let foo() = {
4     // some comment
5     ... do something ...
6   }
7   ```,
8   l => not l.starts-with(regex("\s*//"))
9 )

```

```

let foo() = {
  ... do something ...
}

```

Parameters:

raw-block (content or str) – a single raw element or (multi line) string

test (function) – a function that takes a string and returns a new one

```
slice(raw-block: content str, ..args: arguments) -> content
```

Slices lines of a raw element and creates a new one with the lines. All properties of the element (e.g. block and lang) are preserved.

```

1 crudo.slice(
2   ```typc
3   let foo() = {
4     // some comment
5     ... do something ...
6   }
7   ```,
8   1, 3,
9 )

```

```

// some comment
... do something ...

```

Parameters:

`raw-block (content or str)` – a single raw element or (multi line) string

`..args (arguments)` – the same arguments as accepted by `array.slice()`

```

lines(raw-block: content str, ..line-numbers: arguments, zero-based: boolean)
-> content

```

Extracts lines of a raw element similar to how e.g. printers select page ranges. All properties of the element (e.g. `block` and `lang`) are preserved.

This function is comparable to `slice()` but doesn't have the option to specify the *number* of selected lines via count. On the other hand, multiple ranges of pages can be selected, and indices are one-based by default, which may be more natural for line numbers.

Lines are selected by any number of parameters. Each parameter can take either of three forms:

- a single number: that line is included in the output
- an array of numbers: these lines are included in the output (a major usecase being `range()` – but beware that `range()` uses an exclusive end index)
- a string containing numbers (e.g. "1") and inclusive ranges (e.g. "2-3") separated by commas. Range limits may be omitted (e.g. "-2", "2-"), meaning the range starts/ends at the first/last line. Whitespace is allowed.

All three kinds of parameters can be mixed, and lines can be selected any number of times and in any order.

```

1 crudo.lines(
2   ```typc
3   let foo() = {
4     // some comment
5     ... do something ...
6     // another comment
7   }
8   ```,
9   "-2,4-,1", "2-3", range(3, 5), 5,
10 )

```

```

let foo() = {
  // some comment
  // another comment
}
let foo() = {
  // some comment
  ... do something ...
  ... do something ...
  // another comment
}

```

Parameters:

`raw-block (content or str)` – a single raw element or (multi line) string

`..line-numbers (arguments)` – any number of line number specifiers, as described above

`zero-based (boolean = false)` – whether the supplied numbers are one-based line numbers or zero-based indices

```
join(..raw-blocks, main) -> content
```

Joins lines of multiple raw elements and creates a new one with the lines. All properties of the main element (e.g. block and lang) are preserved.

```
1  crudo.join(typc
2    ``java
3    let foo() = {
4      // some comment
5      ... do something ...
6    }
7    `` ,
8    ``typc
9    let bar() = {
10     // some comment
11     ... do something ...
12   }
13   `` ,
14   main: -1,
15 )
```

```
let foo() = {
  // some comment
  ... do something ...
}
let bar() = {
  // some comment
  ... do something ...
}
```

String parameters are allowed; the main parameter defaults to the first raw block:

```
1  crudo.join(typc
2    "// these strings don't",
3    "// determine the properties",
4    ``typ
5    // this raw block does:
6    // still Typst!
7    `` ,
8  )
```

```
// these strings don't
// determine the properties
// this raw block does:
// still Typst!
```

- `..raw-blocks (arguments)`: any number of single raw elements or (multi line) strings
- `main (int, auto)`: the index of the raw element of which properties should be preserved. Negative indices count from the back. `auto` chooses the first positional argument that is a raw element and not a string, if any.

II.a regions

• `ranges()`

• `extract()`

• `ranges-within()`

```
ranges(raw-block: content str, regions: none str array) -> array
```

Returns an array of line ranges; each range is a pair of numbers representing the lower (inclusive) and upper (exclusive) bound of the range.

The ranges are selected through the specified regions, and regions are indicated in the raw-block by comments of the form `// @region start:<region-name>` and `// @region end:<region-name>` (multiple `start:` and `end:` indicators may be part of the same comment). Regions don't need to be nested; they can freely overlap.

Making the comment format more flexible (e.g. allowing `# @region ...` for Python) is planned but not yet supported.

There is no one to one correspondence between the requested regions and the returned ranges:

- if a region contains region markers, those markers will be excluded, resulting in multiple ranges for that region;
- if a region appears multiple times in the raw-block, each appearance will have its own range(s).

The resulting ranges can be used with `lines()` (although see `extract()` as a shortcut), `ranges-within()`, or with libraries such as `codly` or `zebraw`. See Section I.b for more examples on using regions.

Parameters:

`raw-block (content or str)` – a single raw element or (multi line) string

`regions (none or str or array)` – a single or array of region names, or none to get all lines that don't indicate regions. Prefixing a region with `!` excludes the region instead of including it.

```
extract(raw-block: content str, regions: none str array) -> content
```

Uses `ranges()` combined with `lines()` to select a subset of lines from a code snippet. This function is equivalent to

```
1 lines(raw-block, ..ranges(raw-block, regions).map(((a, b)) => range(a, b))) typc
```

Parameters:

`raw-block (content or str)` – a single raw element or (multi line) string

`regions (none or str or array)` – a single or array of region names, or none to get all lines that don't indicate regions. Prefixing a region with `!` excludes the region instead of including it.

```
ranges-within(outer-ranges: array, inner-ranges: array) -> array
```

Translates the inner - ranges line numbers for use in a code block that has been reduced to only contain the ranges from outer - ranges.

Let's say we have a code snippet with 20 lines and wanted to skip lines 1-2 and 11-12. We would therefore specify outer-ranges as `((3, 11), (13, 21))` (upper bound is exclusive). In the resulting snippet, we want to refer to lines 4 and 14-15, so we specify inner-ranges as `((4, 5), (14, 16))` —but because we're removing lines, we need to use different line numbers to refer to these! `ranges-within()` will give us the correct result ranges of `((2, 3), (10, 12))`.

If the inner-ranges contain lines that are not contained in the outer-ranges, these will be dropped. See Section I.b.a for more examples on using ranges within ranges.

Parameters:

`outer-ranges (array)` – an array of line ranges, as returned by `ranges()`. This determines what lines are present in the resulting code snippet and what line numbers are skipped.

`inner-ranges (array)` – an array of line ranges, as returned by `ranges()`. This determines what lines should be selected within the outer ranges. Lines not inside the outer ranges are skipped, and line numbers are changed to account for lines that don't appear in the outer ranges.

II.b history

- `ranges()`

- `extract()`

- `ranges-within`

```
ranges(raw-block: content str, current: str, history: array) -> array
```

Returns an array of line ranges; each range is a pair of numbers representing the lower (inclusive) and upper (exclusive) bound of the range.

The ranges that are returned are specified via the current step, which must be an element of the specified history of steps. In the raw-block, the steps are indicated by comments of the form `// @start:<step-name>` and `// @end:<step-name>`; steps that remove code are enclosed in `/* @before:<step-name>` and `*/`.

The lines that are returned are those that come in the history before or at the current step, but have not been removed before the current step.

The resulting ranges can be used with `lines()` (although see `extract()` as a shortcut), `ranges-within`, or with libraries such as `codly` or `zebraw`. See Section I.c for more examples on using histories.

Parameters:

`raw-block (content or str)` – a single raw element or (multi line) string

`current (str)` – the step at which to show the file

`history (array)` – an array of step names that appear in the file, establishing the order of modifications to determine what steps have come before current.

```
extract(raw-block: content str, current: str, history: array) -> content
```

Uses `ranges()` combined with `lines()` to select a subset of lines from a code snippet. This function is equivalent to

```
1 lines(raw-block, ..ranges(raw-block, current, history).map((a, b)) =>  
  range(a, b)))
```

typc

Parameters:

`raw-block (content or str)` – a single raw element or (multi line) string

`current (str)` – the step at which to show the file

`history (array)` – an array of step names that appear in the file, establishing the order of modifications to determine what steps have come before current.

`ranges-within:` `function`

This is a re-export of `regions.ranges-within()` since that function is also appropriate for the ranges produced by this module.